GD GOENKA UNIVERSITY
BACHELOR OF TECHNOLOGY · COMPUTER SCIENCE ENGINEERING

FINAL MASTER PRODUCT REQUIREMENTS DOCUMENT

AI-Powered Recruitment & Applicant Tracking System

Consolidated product, architecture, and operational specification for an explainable, auditable AI screening platform

PRODUCT DEFINITION · SYSTEM ARCHITECTURE · AI / ML PIPELINE · EXPLAINABLE AI · SECURITY & COMPLIANCE · OPERATIONS

SUBMITTED BY
Riya Jaiswal

DOCUMENT VERSION
3.0 (Master)

PREPARED FOR
Engineering, Product & Delivery

ACADEMIC YEAR
2026–27

DATE
August 3, 2026

CLASSIFICATION
Internal — Project Use

1. Version History

This master document consolidates two predecessor documents. Where the sources conflicted, the resolution and its rationale are recorded below rather than left implicit.

Table 1.1 — Document version history

VERSION	DATE	AUTHOR	SUMMARY OF CHANGES
0.1	Jul 31, 2026	R. Jaiswal	Initial product draft. Contained explicit working assumptions and eight unresolved discovery questions. Superseded.
1.0	Jul 31, 2026	R. Jaiswal	Product Requirements Document. Business framing, personas, journeys, user stories, functional and non-functional requirements, milestones, risks, acceptance criteria.
2.0	Aug 1, 2026	R. Jaiswal	Development & Production PRD. System architecture, database design, AI/ML pipeline, explainability, infrastructure, CI/CD, security, operations.
3.0	**Aug 3, 2026**	**R. Jaiswal**	**Final Master PRD.** Merged v1.0 and v2.0 by selecting the stronger source per section. Conflicts resolved (see Table 1.2). Added version history, success-metric instrumentation, API specifications, glossary, and references. Removed duplicated scope and business-rule statements that appeared in both sources.

Table 1.2 — Conflict resolution log

#	CONFLICT	V1.0 POSITION	V2.0 POSITION	RESOLUTION & RATIONALE
C1	AI model sourcing	Third-party LLM/embedding APIs for v1	Self-hosted spaCy, sentence-transformers, LightGBM on EKS	**Adopt v2.0.** Self-hosted models are the only option consistent with the SHAP-based per-feature explainability (§22), the offline fairness audit on a restricted protected-attribute dataset (§21.4), and the in-region data-residency requirement (NFR5). Third-party inference APIs would place candidate PII outside the operating region and make feature-level attribution infeasible.
C2	Delivery timeline	3–4 month MVP, bootstrapped	19 weeks across six phases	**Adopt v2.0.** The 19-week plan is decomposed into phase-level durations with dependencies and is consistent with the milestone set; the 3–4 month figure was an unvalidated assumption.
C3	Product naming	Product name marked “TBD”	Referred to descriptively	**Resolved.** Working name fixed as **TalentLens** throughout, eliminating the placeholder.
C4	Compliance jurisdiction	US-first, EU-ready (assumed)	Region-agnostic “operating region”	**Merged.** Retain v2.0’s region-neutral architecture; retain v1.0’s specific regulatory analysis (EEOC, four-fifths rule, GDPR/CCPA) as §24.5, which v2.0 lacked.
C5	Scalability target	500 concurrent sessions, 50k	Same figures, treated as firm with 3×	**Adopt v2.0.** Figures retained and

#	CONFLICT	V1.0 POSITION	V2.0 POSITION	RESOLUTION & RATIONALE
		applications/month (marked assumed)	headroom	promoted to firm requirements with the stated autoscaling headroom.
C6	Auto-rejection policy	Open question — advisory vs. binding	Human review mandatory, no autonomous rejection	**Adopt v2.0.** Aligns with BR-01 and with the regulatory analysis in §24.5. The open question is closed.

2. Table of Contents

§	SECTION	§	SECTION
1	Version History	18	ER Diagram
2	Table of Contents	19	AI/ML Pipeline
3	Executive Summary	20	Resume Parsing Workflow
4	Problem Statement	21	Candidate Ranking Workflow
5	Product Vision	22	Explainable AI Workflow
6	Business Goals	23	API Specifications
7	Success Metrics	24	Security & Privacy
8	User Personas	25	Monitoring & Logging
9	User Journey	26	Testing Strategy
10	User Stories	27	Deployment Strategy
11	Functional Requirements	28	Risk Analysis
12	Non-Functional Requirements	29	Timeline
13	Business Rules	30	Acceptance Criteria
14	Scope	31	Future Scope
15	System Architecture	32	Glossary
16	Technology Stack	33	References
17	Database Design	—	Appendix A — Supplementary Diagrams

3. Executive Summary

TalentLens is an AI-assisted recruitment and applicant tracking platform that helps growing companies source, screen, rank, and manage candidates faster and more consistently than legacy ATS tools — without surrendering human judgment or auditability.

The platform pairs conventional pipeline management with two AI capabilities: structured resume parsing and candidate–job matching. Critically, neither capability operates as a black box. Every score the system produces is accompanied by a human-readable rationale derived from per-feature attribution, is stamped with the model ver-

sion that generated it, and can be overridden by a recruiter at any time. Every AI-influenced decision and every override is written to an append-only audit log.

The product targets small-to-mid-market companies (approximately 50–2,000 employees) that today rely on manual resume screening or keyword-search ATS tools. The primary commercial promise is a **50% reduction in time-to-shortlist**; the primary defensive promise is that the screening process remains transparent, auditable, and testable for adverse impact.

Architecturally, the system is a service-oriented platform of three deployable units — a React web client, a NestJS core API, and a Python/FastAPI AI service — backed by PostgreSQL, Redis, OpenSearch, and S3, deployed on AWS EKS. The AI service is deliberately isolated from the transactional core so that model retraining, promotion, and rollback never require redeploying business logic, and so that a degraded model cannot take down the recruiter workflow.

All AI models are **self-hosted** (spaCy for entity extraction, sentence-transformers for semantic matching, LightGBM for final re-ranking). This is a deliberate decision, not a cost optimisation: feature-level explainability, offline fairness auditing against protected attributes, and in-region data residency are all requirements that third-party inference APIs would compromise.

This document is the single source of truth for the project through design, development, testing, and launch. It is written to serve simultaneously as a B.Tech final-year project submission and as a specification a working development team could build against.

---

4. Problem Statement

Recruiting teams at growing companies spend a disproportionate share of their working hours manually reviewing resumes. Existing ATS platforms in this market segment largely offer keyword search and manual filtering: they provide no intelligent ranking, no explanation of why a candidate surfaced or was missed, and no structured record of how screening decisions were reached.

This produces three compounding failures:

4.1 Slow time-to-hire. Recruiters triage large applicant pools by hand. For a role attracting several hundred applications, first-pass screening alone can consume days of recruiter time, delaying every downstream stage and losing strong candidates to faster-moving competitors.

4.2 Inconsistent screening quality. Manual review quality degrades with reviewer fatigue and varies with subjective judgment. The same resume may be assessed differently depending on when in a review session it is encountered, or by whom. There is no mechanism to detect this inconsistency, because the basis for each decision is never recorded.

4.3 Unmanaged bias risk. Because manual screening decisions are rarely documented, companies cannot demonstrate — to a regulator, a court, or themselves — that their hiring process is free of disparate impact. This is a growing legal exposure as AI-in-hiring regulation expands, and it is equally a risk in *purely manual* processes, which are simply undocumented rather than unbiased.

4.4 The candidate side. Applicants experience this as a slow, opaque process: long application forms requiring re-entry of information already present in their resume, no status visibility after submission, and no explanation of outcomes.

The core insight. Simply adding AI ranking to this picture solves 4.1 while *worsening* 4.3 — an unexplained model score is less auditable than an unexplained human one, because it is applied at scale. The product opportunity is therefore not "AI screening" but **AI screening that is explainable, overridable, and auditable by construction.**

---

5. Product Vision

> *To be the applicant tracking system that growing companies trust to make hiring faster and fairer — combining AI-assisted screening with full transparency, so that every recommendation the system makes can be understood, reviewed, and, when necessary, overridden by a human decision-maker.*

Three commitments follow from this vision and constrain every design decision in this document:

1. **No score without a reason.** A ranking the recruiter cannot interrogate is not shipped. This is enforced at the database layer (§17), the service layer (§22), and the UI layer (§15.4).
2. **The human decides.** AI output is advisory. The system never removes a candidate from consideration autonomously.
3. **Everything is on the record.** Scores, overrides, stage transitions, and administrative changes are written to an append-only audit trail that can be exported and reconciled.

---

6. Business Goals

Table 6.1 — Business goals and targets

ID	GOAL	TARGET	MEASUREMENT BASIS
BG-1	Reduce average time-to-shortlist	50% reduction vs. manual baseline	Baseline captured during design-partner onboarding, before AI scoring is enabled
BG-2	Achieve recruiter adoption	70%+ weekly active usage among licensed seats	Distinct authenticated recruiter sessions per rolling 7 days ÷ provisioned seats
BG-3	Deliver candidate experience quality	80%+ application completion rate	Submitted applications ÷ application forms opened
BG-4	Maintain screening reliability	<5% false-negative rate on qualified candidates	Recruiter-confirmed qualified candidates ranked outside the surfaced shortlist
BG-5	Maintain platform reliability	99.5% uptime	Successful health-check probes ÷ total probes, monthly
BG-6	Sustain fairness compliance	All protected-group outcomes within the four-fifths adverse-impact threshold	Quarterly offline audit (§21.4)

Goals BG-4 and BG-6 are deliberately framed as *ceilings on harm* rather than growth targets. A version of this product that hits BG-1 while failing BG-6 is not a successful product; §30 makes this a hard release gate.

7. Success Metrics

Success metrics are separated from business goals because they are instrumented differently: business goals are reviewed with stakeholders quarterly, whereas the metrics below are computed continuously and surfaced on the internal dashboards described in §25.

Table 7.1 — Product success metrics

METRIC	TYPE	TARGET	INSTRUMENTATION
Time-to-shortlist	**North Star**	–50% vs. baseline	Interval from `jobs.published_at` to first application reaching the Screening stage
Recruiter weekly active usage	Adoption	≥70% of licensed seats	Authenticated session events, deduplicated per user per day
Application completion rate	Candidate UX	≥80%	Funnel events: `application.form_opened` → `application.submitted`
Shortlist precision	AI quality	≥75%	Proportion of AI-surfaced candidates a recruiter advances or marks relevant
Recruiter override rate	AI trust	No significant upward regression release-over-release	Ratio of `score.overridden` events to scores displayed
Parsing field-level F1	AI quality	≥0.90	Nightly evaluation against the frozen golden dataset (§26.7)
Adverse-impact ratio	Fairness	Within four-fifths threshold, all groups	Quarterly offline audit against the restricted fairness dataset
Platform uptime	Reliability	≥99.5%	Route 53 health checks against `/health`
Scoring latency (p95)	Performance	≤400 ms	Prometheus histogram on the AI service scoring endpoint

Note on override rate. A rising override rate is treated as a *quality signal*, not a usage failure. It indicates recruiters increasingly disagree with the model, and it triggers the retraining review described in §19.5. The metric is therefore monitored for regression rather than driven toward zero — a system with a zero override rate would suggest recruiters have stopped exercising the judgment BR-01 requires of them.

8. User Personas

8.1 Recruiter — primary user Manages multiple open requisitions simultaneously, often 8–15 concurrently. Needs fast, trustworthy candidate shortlists, the ability to override AI recommendations without friction, and clear visibility into pipeline status across all active roles. Lives in the product daily; efficiency of repeated actions matters more than onboarding polish. *Success for this persona:* spending screening time on the twenty candidates worth reading rather than the two hundred that arrived.

8.2 Hiring Manager Reviews shortlisted candidates and provides structured feedback. Spends limited time in the platform — often only a few minutes per candidate — and requires simple, focused views plus asynchronous collaboration through comments and scorecards. Should never need to learn the pipeline mechanics. *Success for this persona:* being able to give useful, comparable feedback without a training session.

8.3 Candidate Applies to open roles and expects a fast, low-friction, mobile-friendly application process, transparent status updates, and — where local regulation grants it — the ability to understand the basis of any AI-influenced decision affecting them. Interacts with the product rarely and under stress. *Success for this persona:* applying in under three minutes and never wondering whether the application was received.

8.4 Platform Administrator Manages the company account, user roles and permissions, integrations, and compliance configuration — including the authority to enable or disable AI-assisted scoring per job. Accountable for producing audit evidence on request. *Success for this persona:* being able to answer "how was this candidate assessed?" with an exported record rather than a reconstruction.

Table 8.1 — Persona summary

PERSONA	FREQUENCY OF USE	PRIMARY NEED	KEY RISK IF UNSERVED
Recruiter	Daily, sustained	Trustworthy, fast shortlists	Reverts to manual screening; product unused
Hiring Manager	Weekly, brief	Simple review and feedback	Feedback stays in email; pipeline data incomplete
Candidate	Once per application	Speed and status transparency	Application abandonment (BG-3 failure)
Administrator	Monthly, plus on demand	Control and auditability	Cannot demonstrate compliance

9. User Journey

9.1 Recruiter Journey — filling an open role

1. Creates a new job posting with structured requirements.
2. Publishes the role and monitors incoming applications.
3. Reviews AI-generated candidate rankings alongside their rationale.
4. Adjusts ranking weights for the role, or overrides individual scores, as needed.
5. Moves qualified candidates through pipeline stages.
6. Schedules interviews through the integrated calendar.
7. Collaborates with hiring managers via comments and scorecards.
8. Tracks funnel analytics to identify stage bottlenecks.
9. Records the final hiring decision, closing the requisition.

9.2 Candidate Journey — applying to a role

1. Discovers an open role and opens the application page.
2. Uploads a resume; the system auto-fills structured profile fields.
3. Reviews and corrects any auto-filled information.
4. Submits the application and receives immediate confirmation.
5. Receives status updates as the application progresses.
6. Schedules an interview via a self-service link, if advanced.
7. Receives a final outcome notification.

9.3 Administrator Journey — responding to an audit request

1. Receives a request for evidence covering a defined period and requisition set.
2. Filters the audit log by job, date range, and action type.
3. Exports the record set as CSV or JSON.
4. Reconciles exported scores against the model versions active in that period.
5. Provides the export, with model-version provenance intact, to the requesting party.

This third journey was absent from both source documents. It is included because §11.8 and §30 both make audit export a release gate, and a gate with no described user path is untestable.

---

10. User Stories

Table 10.1 — Prioritised user stories (MoSCoW)

ID	AS A...	I WANT...	SO THAT...	PRIORITY
US-01	Recruiter	an AI match score with a clear rationale	I can identify strong candidates without reading every resume in full	Must
US-02	Recruiter	to override any AI ranking or score	I retain full control over hiring decisions	Must
US-03	Recruiter	a Kanban-style pipeline view	I can track candidates through each stage at a glance	Must
US-04	Recruiter	to adjust ranking weights per job	screening reflects what actually matters for that role	Should
US-05	Hiring Manager	to leave structured feedback on a candidate	my input is captured and visible to the full hiring team	Must
US-06	Candidate	my resume to auto-fill the application form	I can apply quickly without repetitive data entry	Must
US-07	Candidate	to see my application status at any time	I am not left uncertain about where I stand	Must
US-08	Candidate	to correct any mis-parsed field before submitting	an extraction error does not misrepresent me	Must
US-09	Administrator	to disable AI scoring for a specific role	the company can apply manual review where required	Must
US-10	Administrator	an exportable audit log of AI-influenced decisions	the company can demonstrate compliance during an audit	Must
US-11	Recruiter	bulk stage changes across selected candidates	I am not clicking through candidates individually	Should
US-12	Hiring Manager	a comparison view of shortlisted candidates	I can assess them against each other, not in isolation	Could

11. Functional Requirements

11.1 Job Management

ID	REQUIREMENT
FR1.1	Recruiters can create, edit, publish, and close job postings.
FR1.2	Job postings capture structured fields: title, department, location, employment type, required skills, experience level, and description.
FR1.3	Recruiters can duplicate an existing posting as a template.
FR1.4	Each job carries an independent `ai_scoring_enabled` flag, settable only by an Administrator (see BR-04).

11.2 Candidate Application

ID	REQUIREMENT
FR2.1	Candidates apply via resume upload (PDF/DOCX), with structured fields auto-filled from the parsed resume.
FR2.2	Candidates can review and correct any auto-filled field before submission.
FR2.3	Candidates receive confirmation and status-change notifications by email.
FR2.4	The application flow meets WCAG 2.1 AA accessibility standards.
FR2.5	Application submission succeeds and returns confirmation regardless of parsing outcome; parsing failure never blocks intake.

11.3 AI Resume Parsing

ID	REQUIREMENT
FR3.1	The system extracts structured data: contact information, work history, education, skills, and certifications.
FR3.2	A per-field parsing confidence score is generated; fields below the configured threshold are flagged for review rather than silently accepted.
FR3.3	Parsing supports standard PDF and DOCX formats, including single- and multi-column layouts.
FR3.4	Scanned or image-based resumes are routed to OCR fallback rather than rejected.

11.4 AI Matching & Ranking

ID	REQUIREMENT
FR4.1	The system generates a candidate–job match score accompanied by a human-readable rationale describing the top contributing factors.
FR4.2	Recruiters can adjust ranking weights per job (e.g. prioritising skills over years of experience).
FR4.3	Recruiters can manually override any AI-generated rank or score, supplying a reason.
FR4.4	Every AI score is persisted with the associated model version and input feature vector, supporting audit and reproducibility.
FR4.5	Where `ai_scoring_enabled` is false for a job, candidates are presented in reverse-chronological order with no score displayed.

11.5 Pipeline Management

ID	REQUIREMENT
FR5.1	A Kanban-style pipeline view is provided per job, with configurable stages.
FR5.2	Bulk actions are supported (stage change, rejection, tagging) across multiple candidates.
FR5.3	Comments and scorecards are visible to the full hiring team for each candidate.
FR5.4	Every stage transition is recorded with actor and timestamp.

11.6 Interview Scheduling

ID	REQUIREMENT
FR6.1	Calendar integration (Google, Microsoft Outlook) surfaces interviewer availability.
FR6.2	Candidates receive a self-service scheduling link.
FR6.3	If the calendar provider is unavailable, the system degrades gracefully to manual scheduling rather than failing the workflow.

11.7 Analytics

ID	REQUIREMENT
FR7.1	A hiring funnel dashboard displays conversion rates by stage.
FR7.2	Time-to-hire and time-in-stage metrics are tracked per role.
FR7.3	Source-of-hire is tracked and reported.

11.8 Access Control & Administration

ID	REQUIREMENT
FR8.1	Role-based access control is enforced for Administrator, Recruiter, Hiring Manager, and Candidate roles.
FR8.2	Administrators can enable or disable AI scoring per job.
FR8.3	A complete, exportable audit log records all user actions and AI-influenced decisions.
FR8.4	Administrators can revoke all active sessions for a user immediately.

12. Non-Functional Requirements

Table 12.1 — Non-functional requirements

ID	CATEGORY	REQUIREMENT
NFR1	Performance	Resume parsing results returned within 10 s (p95); page loads within 2 s (p95); scoring inference within 400 ms (p95)
NFR2	Availability	99.5% platform uptime, measured monthly
NFR3	Scalability	500 concurrent recruiter sessions and 50,000 applications/month at launch scale, with autoscaling headroom to 3×
NFR4	Accessibility	WCAG 2.1 AA across all candidate- and recruiter-facing flows
NFR5	Data Residency	Candidate and company data stored within the declared operating region; architecture must not preclude additional regions
NFR6	Security	PII encrypted at rest (AES-256/KMS) and in transit (TLS 1.2+); role-based access enforced at the API layer, not only the UI
NFR7	Explainability	No AI-generated score or ranking is displayed without an accompanying rationale
NFR8	Auditability	All AI-influenced decisions retained and exportable for the legally required retention period
NFR9	Recoverability	RTO 4 hours; RPO 15 minutes for the primary datastore
NFR10	Maintainability	≥80% unit-test coverage on business-logic modules; all infrastructure defined as code
NFR11	Observability	Every request traceable end-to-end by correlation ID, including across asynchronous processing

13. Business Rules

These rules are non-negotiable and are enforced in code, not by convention. Each is traced to its enforcement point.

Table 13.1 — Business rules and enforcement

ID	RULE	ENFORCEMENT POINT
BR-01	No candidate may be automatically rejected by the AI system without human review and confirmation.	No API path exists that transitions an application to `Rejected` from a system actor; the stage-transition endpoint requires an authenticated human actor.
BR-02	AI-generated rankings must always be presented with a rationale; a score without an explanation may not be surfaced.	`match_scores.rationale` is `NOT NULL` at the schema level (§17.3); the scoring transaction writes score and rationale atomically.
BR-03	Any recruiter override of an AI score must be logged, including the reason where provided.	`AuditInterceptor` writes before/after state before the override response returns.
BR-04	Administrators, not individual recruiters, control whether AI scoring is enabled for a given requisition.	`RolesGuard` restricts `PATCH /admin/jobs/:id/ai-scoring` to the Administrator role.
BR-05	Candidate data may only be used for the hiring purpose for which it was collected, and must be deletable on request.	Soft-delete via `deleted_at` on candidate-facing entities; erasure routine detaches PII while preserving audit referential integrity.
BR-06	Access to candidate PII is restricted by role; candidates may view only their own application data.	`TenantGuard` scopes all queries by company; `CandidateOwnershipGuard` additionally scopes by `candidate_id`.
BR-07	No protected attribute may be used as a model input feature.	Feature-assembly module has a fixed allow-list; the fairness dataset is stored separately and is never joined into the training feature set.

14. Scope

14.1 In scope — MVP (v1)

- Job posting creation and management
- Candidate application intake with resume upload and auto-fill
- AI-assisted resume parsing into structured candidate profiles
- AI-assisted candidate–job matching and ranking with visible rationale
- Recruiter pipeline management (Applied → Screening → Interview → Offer → Hired/Rejected)

- Manual override of any AI-generated ranking or score
- Interview scheduling via calendar integration
- Recruiter dashboard with hiring funnel analytics
- Role-based access control across four user roles
- Full audit log of AI-influenced decisions, with export

14.2 Out of scope — v1

EXCLUDED	RATIONALE
Automated video or voice interview analysis	Disproportionate compliance and validity risk relative to v1 value
Background and reference check automation	Better served by integration than by first-party build
Native mobile applications	Responsive web meets the candidate need at lower cost
Multi-language resume parsing beyond English	Requires a separate labelled corpus per language
Fully autonomous candidate rejection	Prohibited by BR-01

15. System Architecture

15.1 Architectural principles

1. **Separate transactional and AI workloads.** The recruiter workflow must stay fast and available even while a model is retraining, degraded, or rolling back.
2. **One system of record for PII.** PostgreSQL holds candidate personal data; the search index and embedding store contain derived data only, and can be rebuilt from source.
3. **Stateless compute.** No service holds local state, making horizontal autoscaling and rolling deployment safe by default.
4. **Audit before acknowledge.** An audit record is committed before the originating request returns success — logging is synchronous and non-optional.

15.2 High-level architecture

The platform comprises three deployable units — a React web client, a NestJS core API, and a Python/FastAPI AI service — connected through PostgreSQL, Redis, OpenSearch, S3, and an SQS-backed asynchronous queue.

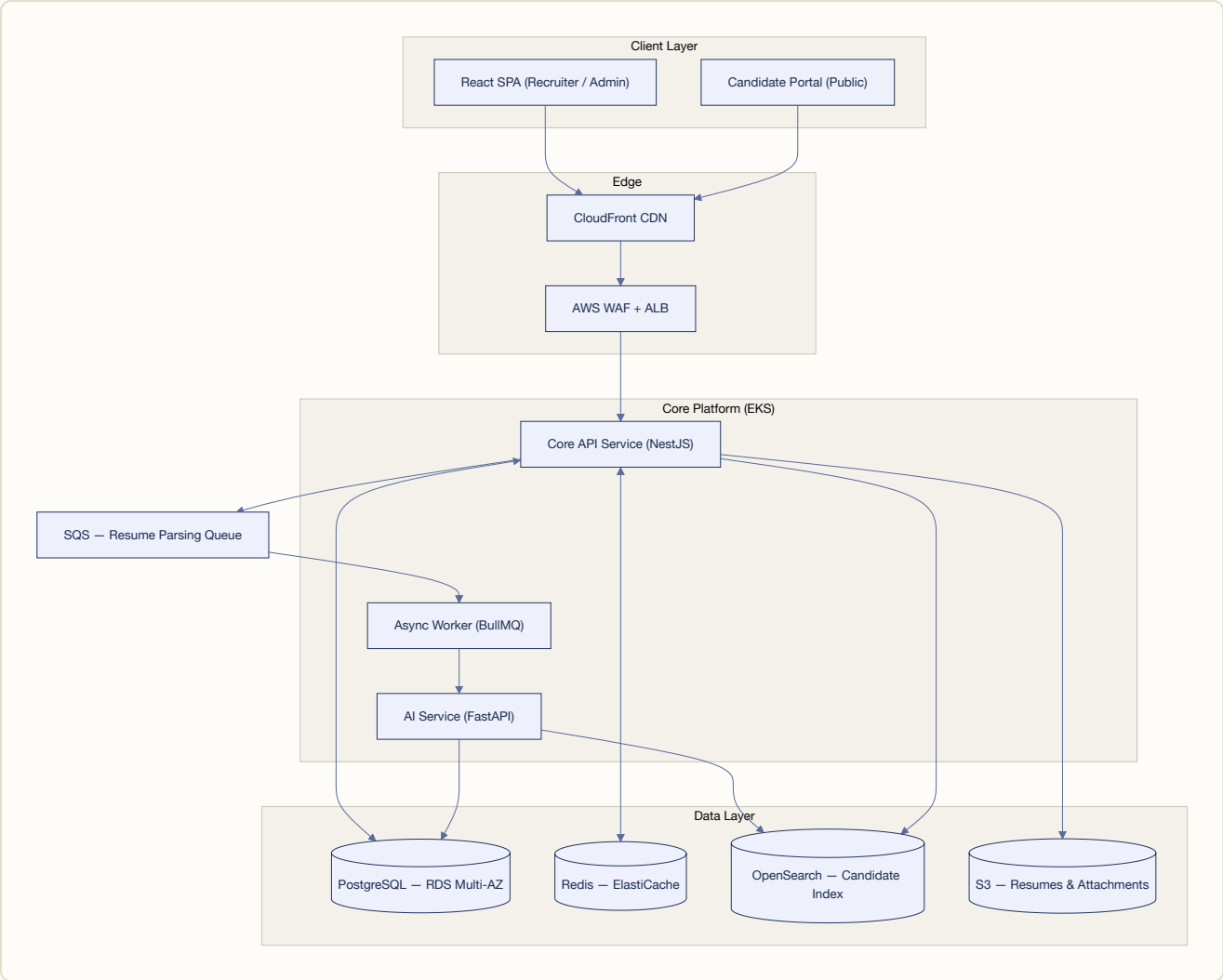


Figure 15.1 — High-level system architecture.

15.3 Low-level architecture — module decomposition

Each service is organised into narrow, single-responsibility modules communicating over well-defined interfaces: REST within a service boundary, domain events across service boundaries.

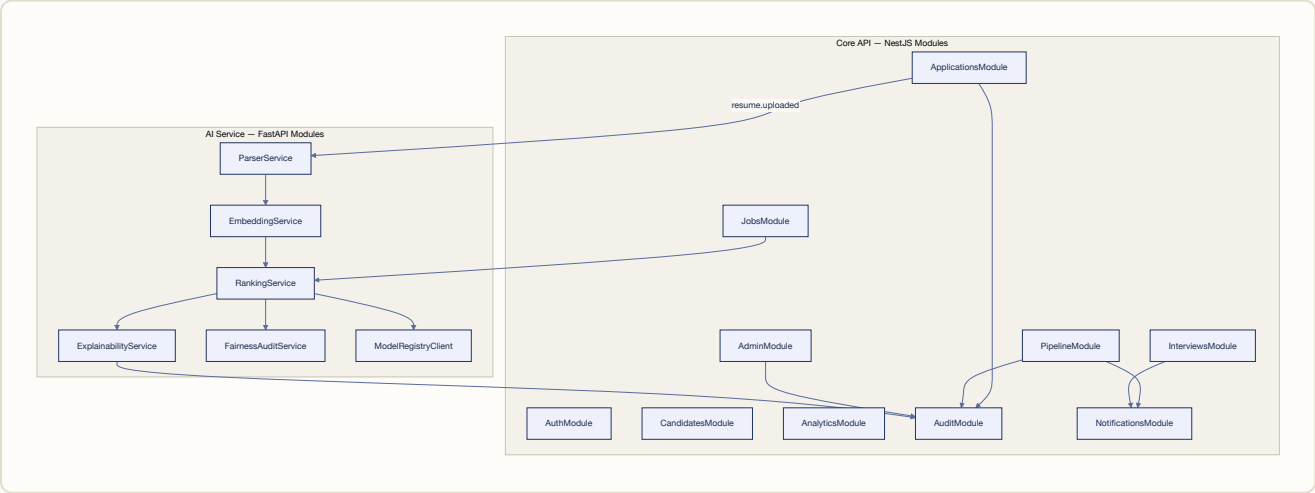


Figure 15.2 — Module decomposition across the Core API and AI Service.

Cross-cutting concerns are implemented as global NestJS interceptors and guards rather than repeated per endpoint:

COMPONENT	RESPONSIBILITY
`AuditInterceptor`	Captures before/after state on every mutating request and commits it to `audit_logs` before the response is sent
`RolesGuard`	Enforces RBAC using metadata declared on each route handler
`TenantGuard`	Scopes every query to the authenticated user's company, preventing cross-tenant access at framework level
`CandidateOwnershipGuard`	Restricts candidate reads to rows they own, independent of role
`ValidationPipe`	Rejects any request body not strictly matching its DTO schema

15.4 End-to-end system flow

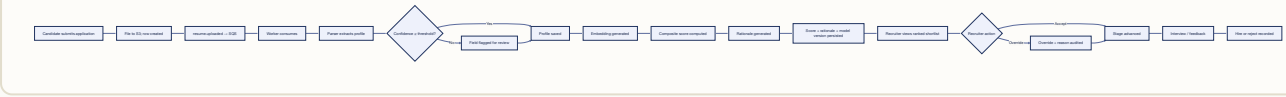


Figure 15.3 — End-to-end flow from application submission to hiring decision.

Invariants enforced along this path:

1. A resume is never scored before parsing confidence is evaluated; low-confidence fields are flagged, never silently dropped.
2. A `MatchScore` row cannot exist without a rationale and a model version (BR-02).
3. An override is recorded as a distinct audit event alongside — not replacing — the original AI score, so both remain queryable.

16. Technology Stack

Table 16.1 — Technology selections and rationale

LAYER	TECHNOLOGY	RATIONALE
Frontend	React 18, TypeScript, Vite, TailwindCSS	Fast build cycle, strong typing, small production bundle
Frontend state	TanStack Query + Zustand	Server-state caching kept separate from lightweight client UI state
Forms & validation	React Hook Form + Zod	One validation schema shared between client and API DTOs
Core API	Node.js 20, NestJS, TypeScript	Modular dependency-injection architecture maps directly onto §15.3 boundaries
ORM	Prisma	Type-safe queries, versioned migrations, schema-as-code
AI Service	Python 3.11, FastAPI	Native access to the ML ecosystem with low serving overhead
NLP / parsing	spaCy custom NER, pdfplumber, Tesseract OCR fallback	Handles both digital and scanned resumes
Embeddings	sentence-transformers (bi-encoder)	Semantic candidate–job similarity beyond keyword overlap
Re-ranking	LightGBM	Gradient-boosted re-ranker; fast, and natively compatible with SHAP
Explainability	SHAP	Per-feature contribution values that drive rationale generation
Primary datastore	PostgreSQL 15 (RDS Multi-AZ)	ACID guarantees required for audit and compliance data
Cache / queue broker	Redis 7 (ElastiCache)	Session cache and BullMQ backing store
Search index	OpenSearch	Candidate search and ranked retrieval at scale
Object storage	Amazon S3 (KMS-encrypted)	Resume files and exported audit archives
Async queue	Amazon SQS + BullMQ workers	Decouples parsing and scoring from the request path
Authentication	JWT (access + rotating refresh), OAuth2/OIDC	Stateless auth with enterprise SSO support
Orchestration	Docker, Kubernetes (Amazon EKS)	Horizontal autoscaling, rolling and canary deployment

LAYER	TECHNOLOGY	RATIONALE
CI/CD	GitHub Actions, Amazon ECR, ArgoCD	GitOps with an auditable promotion history
IaC	Terraform	Reproducible, version-controlled infrastructure
Experiment tracking	MLflow	Model versioning, metric history, promotion gating
Monitoring	Prometheus, Grafana, Sentry	Metrics, dashboards, exception tracking
Logging	Fluent Bit → OpenSearch	Centralised structured logs with correlation IDs
Secrets	AWS Secrets Manager	Rotated credentials; no secrets in source or images

16.1 Repository structure

A pnpm-workspace monorepo, with the Python service as a separate package:

```
talentlens/
├── apps/
│   ├── web/                    # React SPA
│   │   └── src/{features,components,hooks,lib,routes}/
│   ├── api/                    # NestJS core API
│   │   ├── src/modules/{auth,jobs,applications,candidates,
│   │   │                pipeline,interviews,analytics,
│   │   │                admin,audit,notifications}/
│   │   ├── src/common/         # guards, interceptors, decorators
│   │   └── prisma/{schema.prisma,migrations/}
│   └── ai-service/             # FastAPI AI microservice
│       ├── app/{routers,services,models}/
│       └── ml/{training,evaluation,registry}/
├── packages/shared-types/      # DTOs generated from the OpenAPI spec
├── infra/{terraform,k8s}/
├── .github/workflows/
└── docker-compose.yml          # local: postgres, redis, opensearch, localstack
```

17. Database Design

17.1 Conventions

PostgreSQL 15 is the system of record. Tables use `snake_case`; primary keys are UUIDs; every table carries `created_at` and `updated_at`. Candidate-facing entities additionally carry `deleted_at` to support right-to-erasure (BR-05) without breaking referential integrity in the audit trail.

17.2 Schema overview

Table 17.1 — Core entities

TABLE	PURPOSE	KEY COLUMNS
`companies`	Tenant record	`id`, `name`, `data_region`
`users`	Recruiters, hiring managers, administrators	`id`, `company_id`, `email`, `role`, `password_hash`
`jobs`	Job requisitions	`id`, `company_id`, `title`, `required_skills` (jsonb), `ai_scoring_enabled`, `status`
`candidates`	Candidate identity	`id`, `full_name`, `email`, `phone`
`applications`	Candidate ↔ job association	`id`, `job_id`, `candidate_id`, `resume_id`, `current_stage`
`resumes`	Uploaded file metadata	`id`, `application_id`, `s3_key`, `parse_status`
`parsed_profiles`	Structured extraction output	`id`, `resume_id`, `skills` (jsonb), `work_history` (jsonb), `confidence_score`
`match_scores`	AI ranking output	`id`, `application_id`, `model_version_id`, `composite_score`, `rationale` (jsonb)
`ai_model_versions`	Model registry pointer	`id`, `model_name`, `version_tag`, `status`
`pipeline_stages`	Per-job stage configuration	`id`, `job_id`, `name`, `position`
`stage_history`	Stage transition log	`id`, `application_id`, `from_stage`, `to_stage`, `changed_by`
`scorecards`	Structured interviewer feedback	`id`, `application_id`, `author_id`, `rating`, `feedback`
`interviews`	Scheduled interviews	`id`, `application_id`, `scheduled_at`, `calendar_event_id`
`audit_logs`	Append-only action log	`id`, `actor_id`, `action`, `resource_id`, `before_state`, `after_state`

17.3 Compliance-critical DDL

The two tables below carry the platform's core guarantees; their constraints are the enforcement mechanism for BR-02 and BR-03.

```
CREATE TABLE match_scores (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  application_id    UUID NOT NULL REFERENCES applications(id),
  model_version_id  UUID NOT NULL REFERENCES ai_model_versions(id),
  composite_score   NUMERIC(5,4) NOT NULL
                      CHECK (composite_score BETWEEN 0 AND 1),
  rationale         JSONB NOT NULL,   -- BR-02: no score without a reason
  feature_vector    JSONB NOT NULL,   -- FR4.4: reproducibility
  overridden_by     UUID REFERENCES users(id),
  override_reason   TEXT,
  scored_at         TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE audit_logs (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  actor_id      UUID NOT NULL REFERENCES users(id),
  action        TEXT NOT NULL,
  resource_type TEXT NOT NULL,
  resource_id   UUID NOT NULL,
  before_state  JSONB,
  after_state   JSONB,
  correlation_id TEXT NOT NULL,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- The application role holds INSERT and SELECT only.
-- Append-only is enforced by grant, not by application convention.
REVOKE UPDATE, DELETE ON audit_logs FROM application_role;
```

17.4 Indexing strategy

Table 17.2 — Index plan

INDEX	TYPE	QUERY SERVED
`applications(job_id, current_stage)`	Composite B-tree	Pipeline board load
`match_scores(application_id, scored_at DESC)`	Composite B-tree	Latest score per application
`jobs(required_skills)`	GIN	Skill-based job lookup
`match_scores(rationale)`	GIN	Rationale search during audit
`resumes(parse_status) WHERE parse_status <> 'completed'`	Partial	Review queue; stays small as the table grows
`audit_logs(resource_id, created_at DESC)`	Composite B-tree	Per-resource audit reconstruction

18. ER Diagram

The entity-relationship model below reflects the schema in §17. Three relationships carry particular design weight:

- **`APPLICATION → MATCH_SCORE` is one-to-many, not one-to-one.** Re-scoring after a weight change or model promotion appends a new row rather than mutating the existing one, preserving the historical record required by FR4.4.
- **`MATCH_SCORE → AI_MODEL_VERSION` is mandatory.** A score cannot exist without provenance.
- **`AUDIT_LOG` references applications and users but is never cascade-deleted.** Erasing a candidate detaches PII while leaving the audit skeleton intact (BR-05).

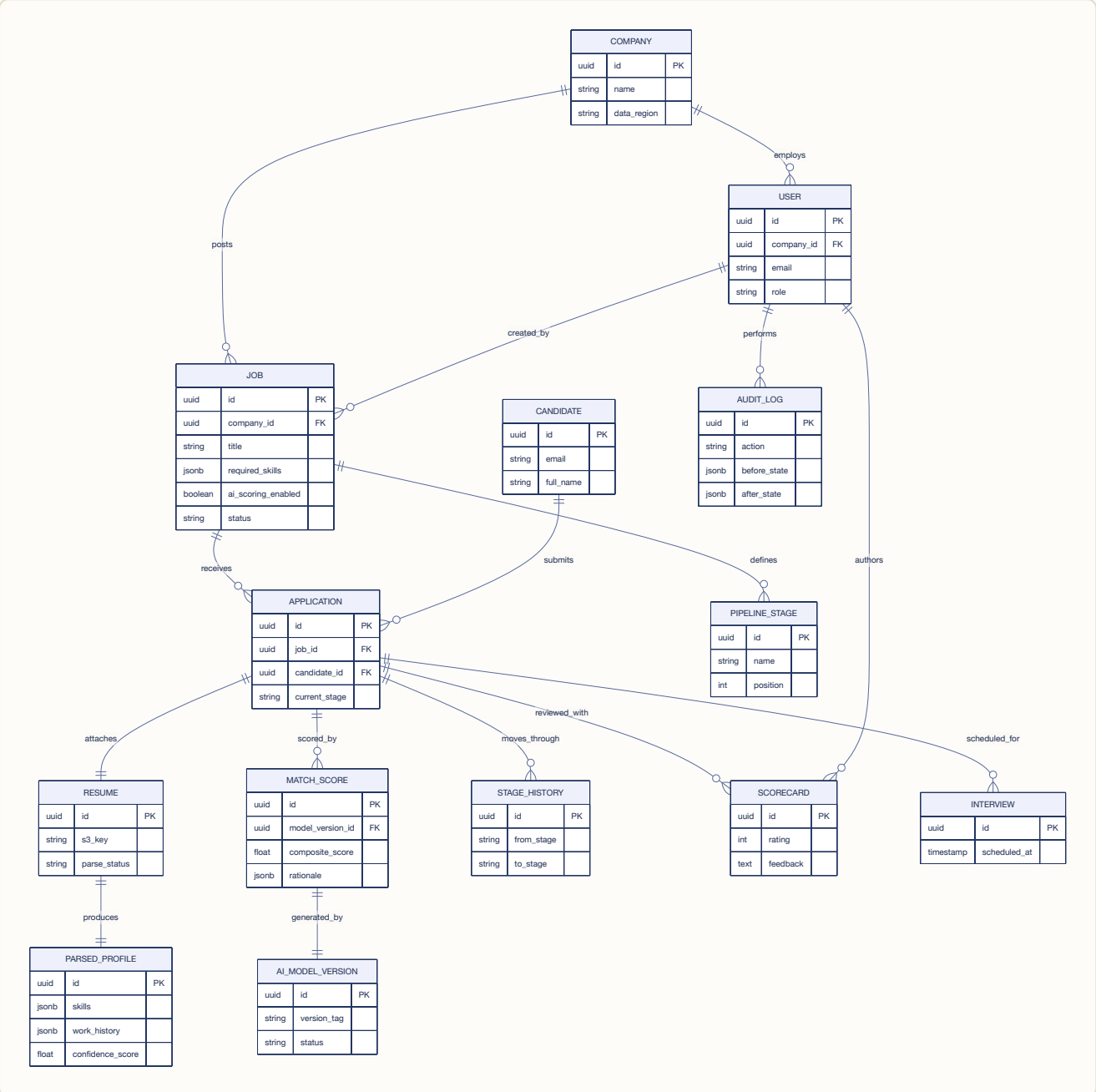


Figure 18.1 — Entity-relationship diagram.

19. AI/ML Pipeline

19.1 Serving architecture

The AI service is a stateless FastAPI application, scaled independently of the core API, backed by an MLflow model registry rather than models baked into the container image. A new model version is promoted by updating a registry pointer — not by rebuilding and redeploying the service.

COMPONENT	RESPONSIBILITY
`ParserService`	Resume text extraction and named-entity recognition
`EmbeddingService`	Bi-encoder inference producing candidate and job vectors
`RankingService`	Composite scoring from semantic similarity plus weighted features
`ExplainabilityService`	SHAP attribution and rationale generation
`FairnessAuditService`	Offline adverse-impact monitoring, stratified by protected attribute
`ModelRegistryClient`	Resolves the active model version at startup and on promotion signal

Models are warmed into memory at pod start. A Redis pub/sub `model.promoted` signal triggers a hot reload across all AI pods without a restart, keeping the staged rollout in §19.6 fast.

19.2 Dataset strategy

ASPECT	APPROACH
Training sources	Consent-gated, PII-scrubbed historical application data; synthetic resume–job pairs covering under-represented skill categories; a licensed public resume corpus for NER pretraining
Primary labels	Recruiter accept/reject/hire decisions
Secondary labels	Human-verified NER field extraction on a sampled subset
Governance	No protected attribute is a model input (BR-07). A separate, access-restricted fairness dataset retains protected-attribute labels *solely* for the offline audit in §21.4 and is never joined into the training feature set
Retention	Training snapshots versioned and retained on the audit-log retention schedule, so any deployed model's training data can be reconstructed

19.3 Feature set

Table 19.1 — Model input features

FEATURE	TYPE	SOURCE
Semantic similarity	Continuous [0,1]	Cosine similarity of candidate and job embeddings
Skill overlap ratio	Continuous [0,1]	Canonical-taxonomy intersection ÷ required skills
Years-of-experience delta	Continuous	Candidate total relevant years – role requirement
Education-level match	Ordinal	Mapped against role requirement
Recency of relevant experience	Continuous	Months since most recent role matching required skills
Certification match count	Discrete	Intersection with role-listed certifications

Explicitly excluded: name, gender, age, date of birth, address, nationality, institution prestige ranking, and photograph. Institution *name* is excluded because it functions as a socio-economic proxy.

19.4 Training pipeline

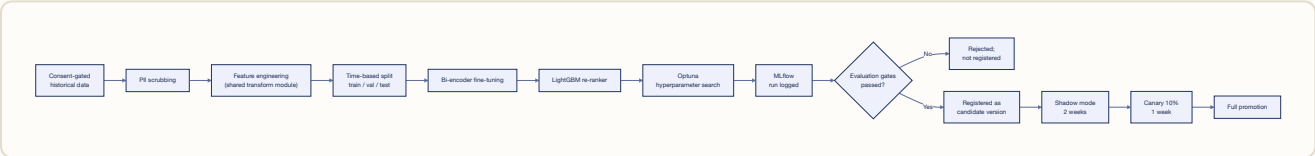


Figure 19.1 — Model training and promotion pipeline.

Key design decisions:

- **Time-based splitting, not random.** A random split would leak future information into training and inflate offline metrics relative to real deployment conditions.
- **A shared feature-transformation module** is used by both training and inference, eliminating train/serve skew by construction rather than by discipline.
- **Every run is logged to MLflow** with hyperparameters, metrics, and artefact lineage.

19.5 Retraining triggers

TRIGGER	CADENCE / CONDITION
Scheduled retraining	Quarterly
Fairness degradation	Any protected-group ratio approaching the four-fifths threshold
Score distribution drift	Population stability index above the configured bound
Override-rate regression	Statistically significant sustained increase (§7)

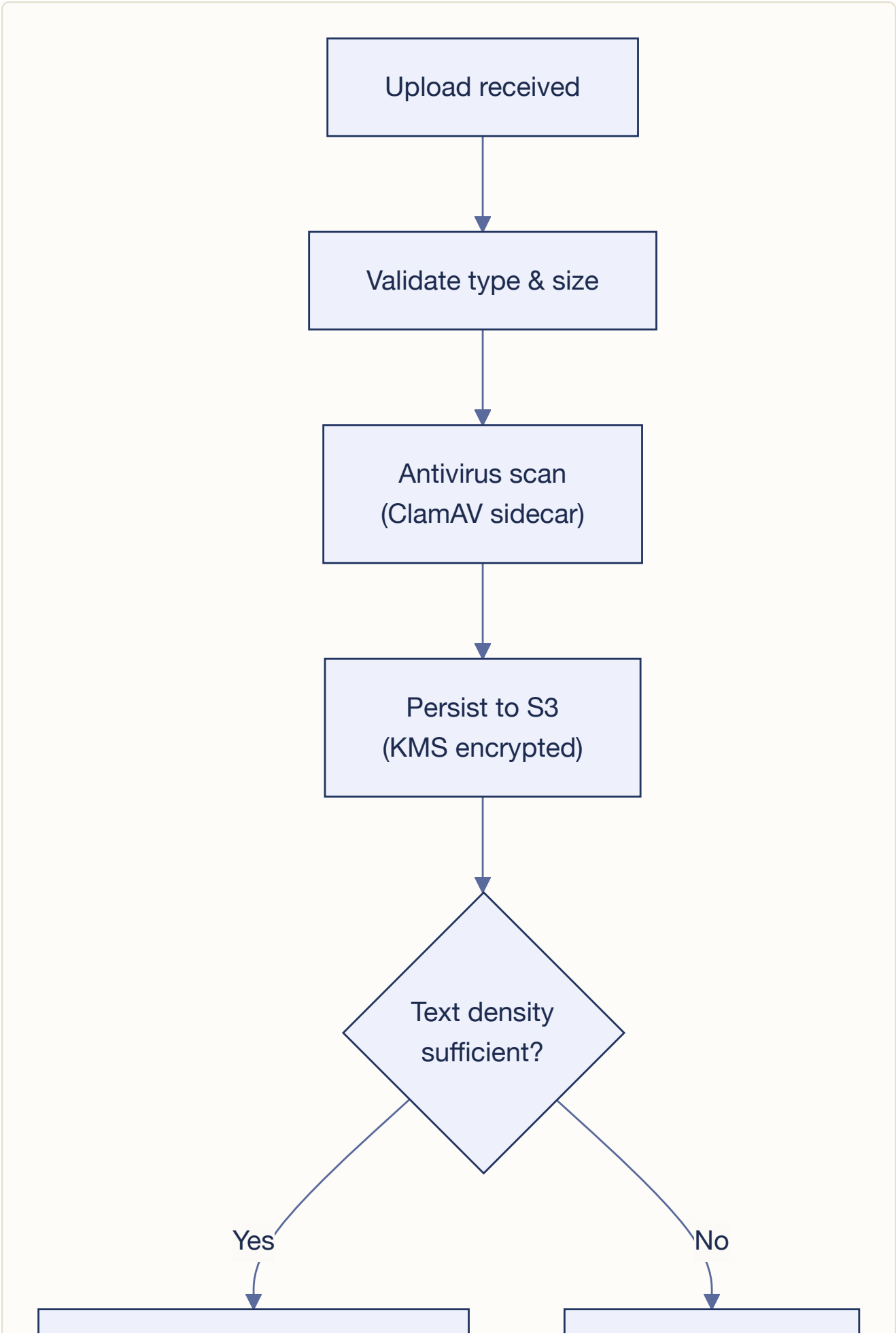
19.6 Evaluation and promotion gates

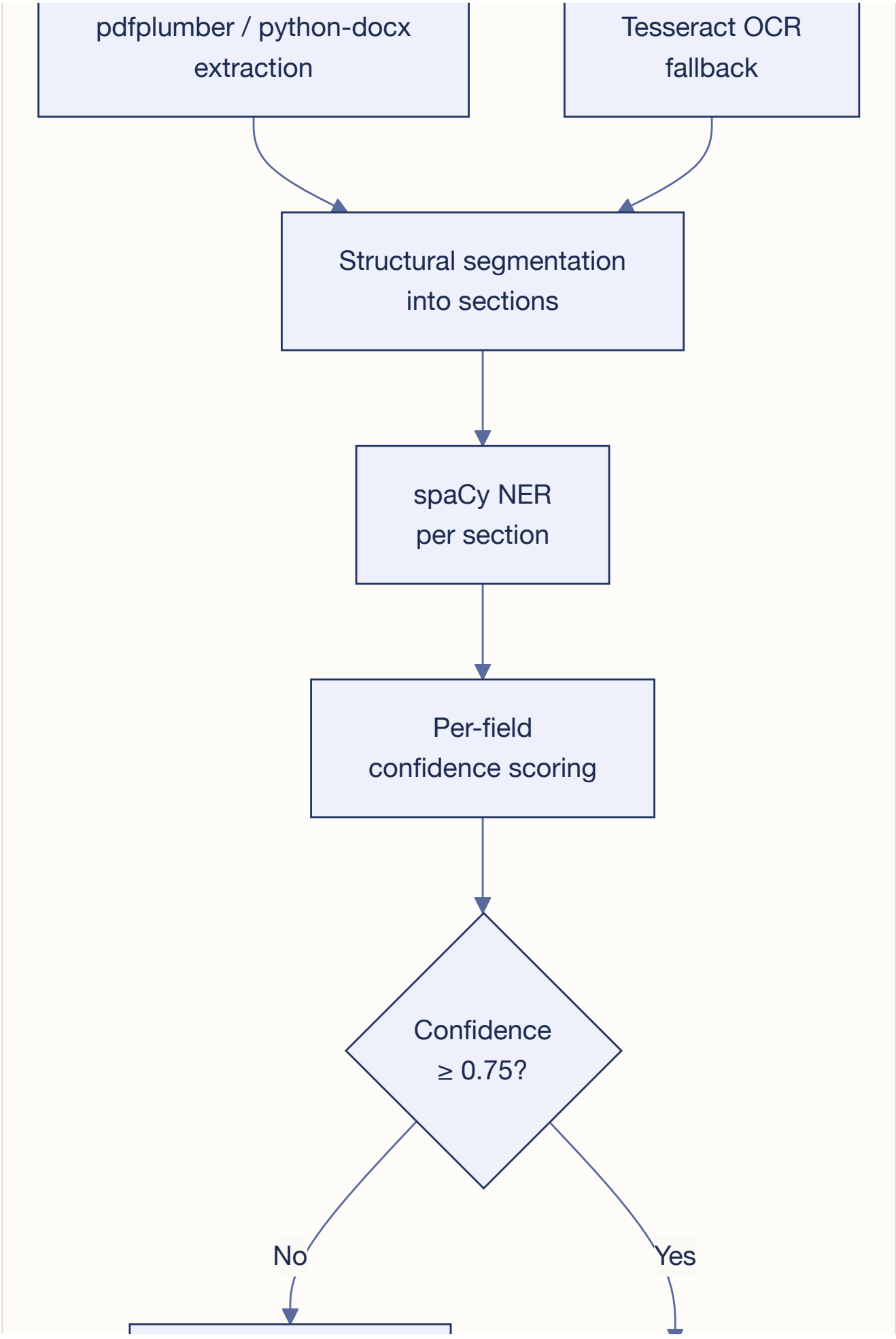
Table 19.2 — Promotion gates

METRIC	PURPOSE	GATE
Precision@10 / NDCG@10	Ranking quality vs. recruiter-confirmed relevance	≥ current production model
Parsing field-level F1	NER extraction accuracy per field type	≥0.90 on the golden dataset
Four-fifths adverse-impact ratio	Fairness across protected groups	Within threshold, **all** groups
Demographic parity difference	Secondary fairness signal	Below configured tolerance
Recruiter override rate	Production trust proxy	No significant regression
Inference latency (p95)	Serving cost	≤400 ms

A failing fairness gate blocks promotion regardless of ranking-quality performance. There is no override path for this gate.

20. Resume Parsing Workflow


Upload received
Validate type & size
Antivirus scan
(ClamAV sidecar)
Persist to S3
(KMS encrypted)
Text density
sufficient?
Yes
No


pdfplumber / python-docx extraction
Tesseract OCR fallback
Structural segmentation into sections
spaCy NER per section
Per-field confidence scoring
Confidence ≥ 0.75?
No
Yes

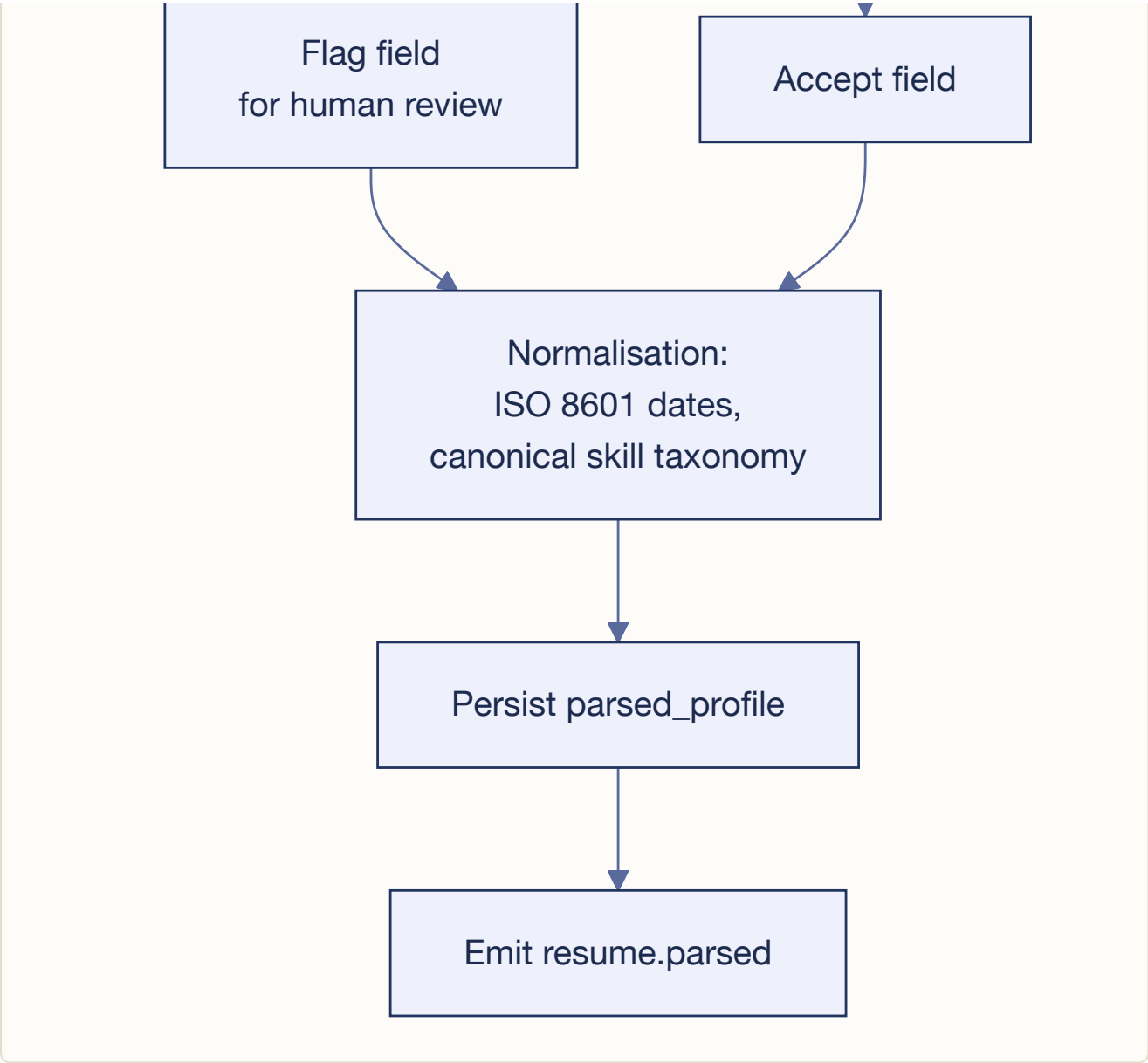


Figure 20.1 — Resume parsing pipeline.

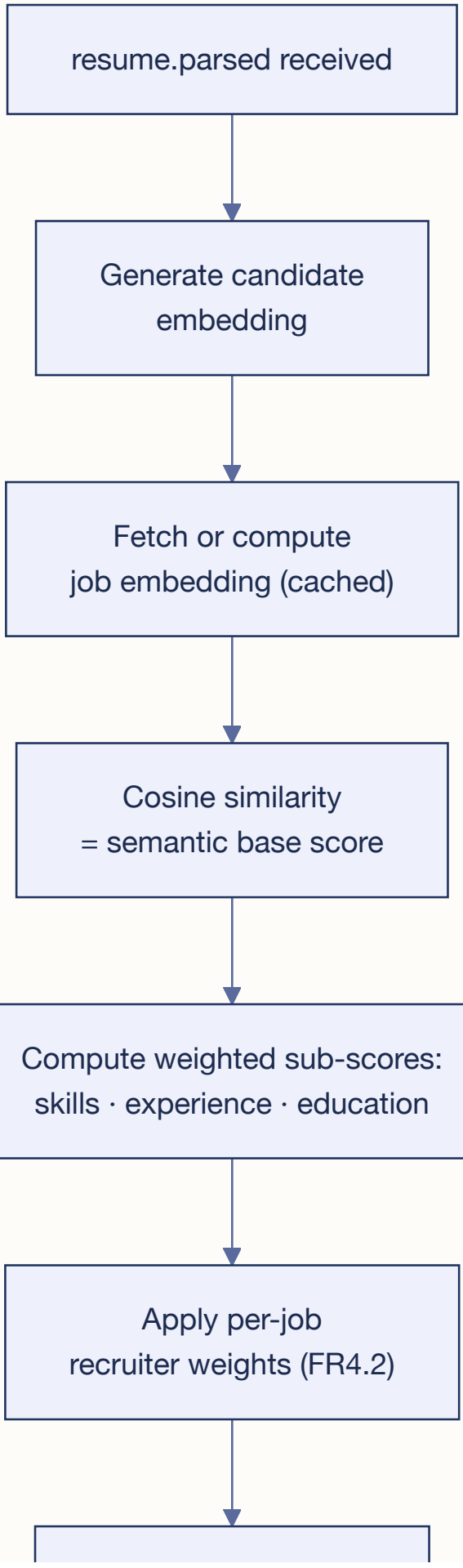
Stage detail:

#	STAGE	DETAIL
1	Upload & validate	File type and size checks; antivirus scan before persistence
2	Text extraction	`pdfplumber` for digital PDFs, `python-docx` for DOCX; documents below a text-density threshold route to OCR
3	Structural segmentation	A lightweight classifier splits raw text into contact/experience/education/skills sections *before* NER, materially improving field-level precision over whole-document extraction
4	Entity extraction	spaCy pipeline fine-tuned on a labelled resume corpus
5	Confidence scoring	Per-field confidence; fields below 0.75 flagged rather than accepted (FR3.2)
6	Normalisation	Dates to ISO 8601; skills mapped to a canonical taxonomy so that "JS", "JavaScript", and "ECMAScript" match consistently
7	Persistence	Written to `parsed_profiles`; `resume.parsed` triggers embedding generation

Design note. Parsing is asynchronous and never blocks application submission (FR2.5). A candidate receives confirmation immediately; if parsing subsequently fails, the application remains valid and is surfaced to the recruiter with a manual-entry prompt. This ordering matters: coupling intake to parsing would make BG-3 (completion rate) hostage to model reliability.

21. Candidate Ranking Workflow

21.1 Scoring sequence


resume.parsed received
Generate candidate embedding
Fetch or compute job embedding (cached)
Cosine similarity = semantic base score
Compute weighted sub-scores: skills · experience · education
Apply per-job recruiter weights (FR4.2)

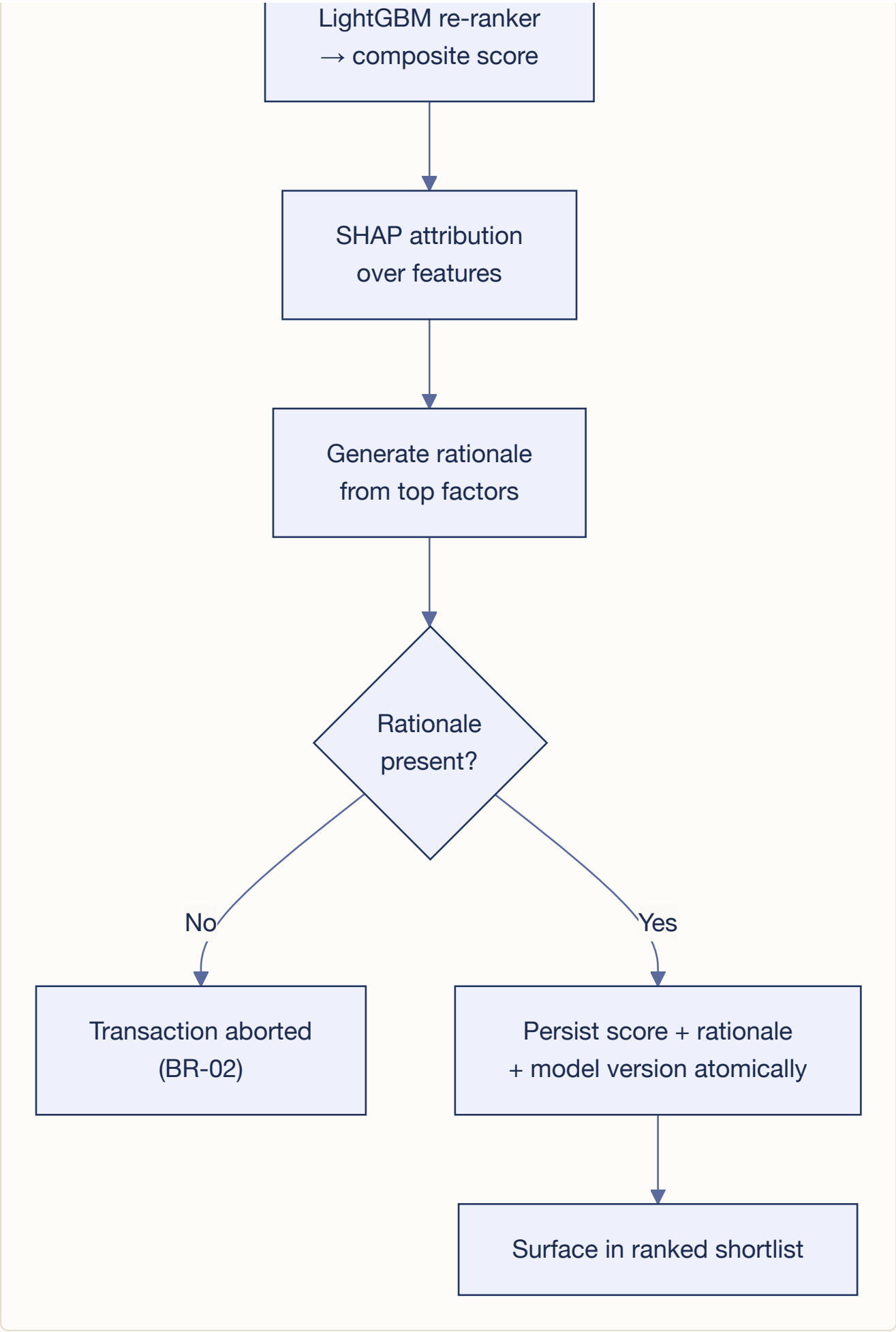

LightGBM re-ranker
→ composite score
SHAP attribution
over features
Generate rationale
from top factors
Rationale
present?
No
Yes
Transaction aborted
(BR-02)
Persist score + rationale
+ model version atomically
Surface in ranked shortlist

Figure 21.1 — Candidate ranking and scoring workflow.

21.2 Steps

1. Generate a candidate embedding from the parsed profile using the sentence-transformer bi-encoder.
2. Retrieve the cached job embedding, or compute it if the job was edited since last scoring.
3. Compute cosine similarity as the semantic base score.
4. Compute weighted sub-scores for skill overlap, experience match, and education match.
5. Apply the recruiter-configured weights for that specific job (FR4.2).
6. Combine into a calibrated composite score in [0,1] via the LightGBM re-ranker trained on historical recruiter decisions.
7. Pass feature contributions to `ExplainabilityService` for rationale generation.
8. Persist score, rationale, feature vector, and model version **atomically**; the write is rejected if the rationale is absent.

21.3 Recruiter controls

CONTROL	EFFECT
Per-job weight adjustment	Re-scores the pool against the new weighting; appends new `match_scores` rows
Individual score override	Replaces displayed rank; original AI score retained and both are auditable
AI scoring disabled (Administrator)	Suppresses all scores for the job; list reverts to reverse-chronological (FR4.5)

21.4 Fairness auditing

The offline audit runs against the access-restricted fairness dataset described in §19.2 — never against production feature data, which by BR-07 contains no protected attributes.

CHECK	METHOD	THRESHOLD
Adverse impact	Selection rate of each protected group ÷ selection rate of the highest-selected group	≥0.80 (four-fifths rule)
Demographic parity difference	Absolute difference in positive-outcome rates	Below configured tolerance
Intersectional check	Adverse impact across combined attribute pairs	≥0.80 where sample size permits statistical significance

Audits run on every model promotion and quarterly in production, independently of any promotion event. Results are presented to the compliance stakeholder and retained with the audit record.

22. Explainable AI Workflow

22.1 Method

SHAP (SHapley Additive exPlanations) values are computed per feature for every scored application. SHAP is chosen over simpler feature-importance methods because it produces *per-prediction* attributions — the explanation is specific to that candidate against that job, rather than a global statement about the model.

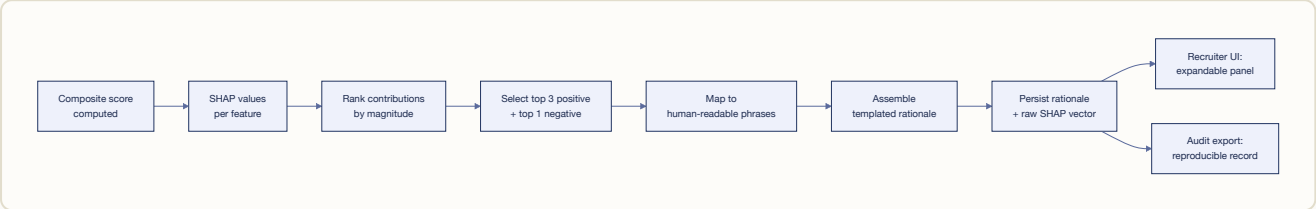


Figure 22.1 — Explainable AI workflow.

22.2 Rationale generation

The top three positive and top one negative contributing factors are mapped to templated, human-readable phrasing. An illustrative output:

> *"Strong match on required skills (Python, SQL) and directly relevant recent work history; limited match on required years of experience."*

Templating rather than free-text generation is deliberate. A generated natural-language explanation could drift from the actual attribution values — producing a plausible sentence that does not reflect what the model did. Templating binds the sentence to the SHAP vector, which is the property that makes the explanation defensible in an audit.

22.3 Enforcement and auditability

PROPERTY	MECHANISM
No score without rationale	`ExplainabilityService` runs synchronously inside the scoring transaction; `rationale` is `NOT NULL`
Reproducibility	Raw SHAP vector, feature vector, and model version stored alongside the rationale
UI-level guarantee	Every score row in the recruiter view exposes an expandable rationale panel; a score cannot render without it
Candidate-facing explanation	The stored rationale is the source for any explanation provided to a candidate on request (§24.5)

23. API Specifications

23.1 Conventions

The OpenAPI 3.0 specification is generated from NestJS decorators via `@nestjs/swagger` and published at `/api/docs`. The `shared-types` package is generated from that specification, so the frontend never hand-writes response types. All endpoints are versioned under `/v1`.

CONVENTION	RULE
Authentication	Bearer JWT in the `Authorization` header; refresh token in an `httpOnly` cookie
Pagination	Cursor-based on all list endpoints; `?cursor=&limit=`
Errors	RFC 7807 problem+json: `type`, `title`, `status`, `detail`, `instance`
Idempotency	`Idempotency-Key` header supported on `POST /applications`
Rate limiting	Per-IP and per-account, enforced at the ALB and API layers

23.2 Endpoint reference

Table 23.1 — Public API endpoints

METHOD	ENDPOINT	DESCRIPTION	AUTHORISED ROLES
`POST`	`/v1/auth/login`	Password or SSO login; returns access + refresh token	Public
`POST`	`/v1/auth/refresh`	Rotates refresh token, issues new access token	Valid refresh cookie
`POST`	`/v1/auth/logout`	Revokes the presented refresh token	Authenticated
`POST`	`/v1/jobs`	Create job posting	Recruiter, Admin
`GET`	`/v1/jobs`	List jobs for the tenant	Recruiter, HM, Admin
`PATCH`	`/v1/jobs/:id`	Edit, publish, or close a job	Recruiter, Admin
`POST`	`/v1/jobs/:id/duplicate`	Duplicate as template (FR1.3)	Recruiter, Admin
`POST`	`/v1/applications`	Submit application with resume	Public (candidate form)
`GET`	`/v1/applications/:id`	Retrieve own application status	Candidate (owner only)
`GET`	`/v1/jobs/:id/candidates`	Ranked candidate list with score and rationale	Recruiter, HM, Admin
`PATCH`	`/v1/applications/:id/stage`	Move application to a new stage	Recruiter, Admin
`PATCH`	`/v1/applications/:id/score`	Override AI score (reason required)	Recruiter, Admin
`POST`	`/v1/applications/:id/scorecards`	Submit structured feedback	Recruiter, HM, Admin
`POST`	`/v1/interviews`	Schedule interview, sync to calendar	Recruiter, Admin
`GET`	`/v1/analytics/funnel`	Hiring funnel conversion metrics	Recruiter, Admin
`GET`	`/v1/audit-logs/export`	Export audit trail (CSV/JSON)	Admin
`PATCH`	`/v1/admin/jobs/:id/ai-scoring`	Enable or disable AI scoring for a job	Admin

Table 23.2 — Internal service-to-service endpoints

METHOD	ENDPOINT	DESCRIPTION	ACCESS
`POST`	`/internal/ai/parse`	Parse resume into structured profile	Cluster-internal, per-request service JWT
`POST`	`/internal/ai/rank`	Score an application against its job	Cluster-internal, per-request service JWT
`POST`	`/internal/ai/explain`	Return SHAP attribution for a score	Cluster-internal, per-request service JWT

Internal endpoints are not exposed through the public ALB listener and are reachable only within the cluster network.

23.3 Representative payloads

Ranked candidate list — `GET /v1/jobs/:id/candidates`

```
{
  "data": [
    {
      "applicationId": "8f14e45f-ceea-467a-9c1e-4b8a1f2d3c44",
      "candidate": { "fullName": "A. Sharma", "email": "a.sharma@example.com" },
      "currentStage": "Applied",
      "score": {
        "composite": 0.8734,
        "modelVersion": "ranker-v2.3.1",
        "rationale": {
          "summary": "Strong match on required skills (Python, SQL) and relevant recent work history
          "factors": [
            { "feature": "skill_overlap",        "contribution":  0.31, "direction": "positive" },
            { "feature": "semantic_similarity",  "contribution":  0.24, "direction": "positive" },
            { "feature": "recency_relevant_exp", "contribution":  0.12, "direction": "positive" },
            { "feature": "years_experience_delta","contribution": -0.09, "direction": "negative" }
          ]
        },
        "overridden": false
      }
    }
  ],
  "nextCursor": "eyJvZmZzZXQiOjI1fQ"
}
```

Score override — `PATCH /v1/applications/:id/score`

```
{
  "overrideScore": 0.95,
  "reason": "Candidate led an equivalent migration at prior employer; resume understates depth."
}
```

The response returns the updated record with `overridden: true`, `overriddenBy`, and the original AI score preserved under `originalScore` — both remain queryable, per §15.4 invariant 3.

24. Security & Privacy

24.1 Authentication

CONTROL	IMPLEMENTATION
Password storage	Argon2id, minimum 12-character policy, breach-list check at signup
Session tokens	15-minute JWT access token; 7-day rotating refresh token in an `httpOnly`, `Secure`, `SameSite=Strict` cookie
Refresh rotation	Previous token invalidated on use, making replay of a stolen token detectable
SSO	OAuth2/OIDC against Google and Microsoft Entra ID
MFA	TOTP required for Administrator; optional for Recruiter and Hiring Manager
Session revocation	All refresh tokens for a user revocable immediately from the admin panel (FR8.4)

24.2 Authorisation matrix

Table 24.1 — Role permissions

RESOURCE / ACTION	RECRUITER	HIRING MANAGER	CANDIDATE	ADMIN
Create / edit jobs	✓	—	—	✓
View AI rankings + rationale	✓	✓ (assigned jobs)	—	✓
Override AI score	✓	—	—	✓
Move pipeline stage	✓	—	—	✓
Submit scorecard	✓	✓	—	✓
View own application status	—	—	✓	—
Enable / disable AI scoring	—	—	—	✓
Export audit log	—	—	—	✓
Manage users and roles	—	—	—	✓

Authorisation is enforced at the API layer (NFR6). Row-level scoping is applied independently of role: `TenantGuard` for company isolation, `CandidateOwnershipGuard` for candidate self-access.

24.3 Data protection

CONTROL	IMPLEMENTATION
Encryption in transit	TLS 1.2+ everywhere, including service-to-service
Encryption at rest	AES-256 via AWS KMS for RDS, S3, and EBS
Secrets	AWS Secrets Manager, 90-day automatic rotation; never committed to source or baked into images
Network isolation	Private subnets for EKS, RDS, and ElastiCache; public subnets only for ALB and NAT
IAM	Least-privilege roles per Kubernetes service account (IRSA); no long-lived AWS credentials in configuration
PII in logs	Custom lint rules (ESLint + pytest plugin) block raw PII in log statements; only resource IDs are logged

24.4 Application security

OWASP Top 10 mitigations: parameterised queries via Prisma (injection); CSP headers plus React's default output escaping (XSS); CSRF tokens on cookie-authenticated state-changing requests; strict DTO validation (mass assignment); rate limiting (brute force). Dependency vulnerability scanning runs on every CI execution; third-party penetration testing is performed annually and after any major architectural change.

24.5 Regulatory considerations

This section is retained from PRD v1.0, whose regulatory analysis was substantially stronger than v2.0's. It is informational and does not constitute legal advice; formal legal review is a release gate (§30).

REGIME	OBLIGATION	PRODUCT RESPONSE
EEOC / anti-discrimination law	AI screening must not produce disparate impact across protected classes	Mandatory pre-launch and quarterly four-fifths testing (§21.4); promotion blocked on failure
Local AI-hiring statutes (e.g. NYC Local Law 144-style)	May require independent bias audit and candidate notice where AI is used in screening	Audit-ready exports (§9.3); per-job AI disclosure; per-job AI disable (FR8.2)
GDPR / CCPA	Right to explanation, right to erasure, data minimisation, consent for automated decision-making	Stored rationale supports explanation requests (§22.3); soft-delete erasure routine (BR-05); protected attributes excluded from features (BR-07)
Emerging human-in-the-loop requirements	Prohibition on solely automated adverse decisions	BR-01: no autonomous rejection path exists in the API surface

25. Monitoring & Logging

25.1 Metrics

Prometheus scrapes `/metrics` on the core API and AI service. Grafana provides per-service dashboards plus a dedicated **AI Health** dashboard.

Table 25.1 — Key monitored signals

SIGNAL	DASHBOARD	ALERT CONDITION
Request rate, latency histogram, error rate	Service	p95 latency breaches NFR1 for 5 min
Queue depth (SQS)	Service	Depth exceeds worker drain rate for 10 min
Parsing throughput and failure rate	AI Health	Failure rate >5% over 15 min
Model inference latency	AI Health	p95 >400 ms
Score distribution drift	AI Health	PSI above configured bound
Recruiter override rate	AI Health	Sustained significant increase
Uptime	Reliability	2 consecutive `/health` failures

25.2 Logging

ASPECT	APPROACH
Format	Structured JSON — `pino` (Node.js), `structlog` (Python)
Correlation	A `correlation_id` propagates from the originating HTTP request through SQS into asynchronous worker and AI processing, so a single candidate's parsing job is traceable end-to-end (NFR11)
Pipeline	Fluent Bit sidecar ships container logs to OpenSearch
Retention	Application logs 90 days hot, 1 year cold (S3). `audit_logs` are excluded from log rotation entirely and retained on the regulatory schedule in their own table with no DELETE grant
PII	Excluded by lint enforcement (§24.3)

25.3 Alerting and incident response

Grafana alert rules route to PagerDuty (paging) and Slack (informational) by severity tier. Sentry captures unhandled exceptions tagged with release and commit SHA, so a spike traces to the deployment that introduced it. Every production incident receives a written postmortem within five business days, with action items tracked to closure.

26. Testing Strategy

Table 26.1 — Test layers

#	LAYER	TOOLING	COVERAGE TARGET
26.1	Unit	Jest (web, api), pytest (ai-service)	≥80% on business-logic modules
26.2	Integration	Testcontainers + supertest (API ↔ Postgres/Redis)	All module boundaries
26.3	Contract	Pact	Web ↔ API, API ↔ AI Service
26.4	End-to-end	Playwright	Apply → parse → rank → stage move → schedule
26.5	Load	k6	p95 latency and throughput at 3× launch scale
26.6	Accessibility	axe-core in CI + manual screen-reader pass	WCAG 2.1 AA on all candidate and recruiter flows
26.7	AI regression	Golden-dataset comparison	No metric regression vs. §19.6 gates on any model PR
26.8	Fairness	Offline adverse-impact suite	Run on every promotion, not only at launch
26.9	Security	Trivy (images), `npm audit` / `pip-audit` (dependencies)	Zero unresolved high/critical

Business-rule test coverage. Each rule in §13 has at least one dedicated failing-path test: attempting to write a `match_score` without a rationale must fail (BR-02); a recruiter token calling the AI-scoring toggle must receive 403 (BR-04); a candidate token requesting another candidate's application must receive 404, not 403, to avoid existence disclosure (BR-06).

27. Deployment Strategy

27.1 Environments

`dev`, `staging`, and `production` run in **isolated AWS accounts** under AWS Organizations — not merely separate namespaces — so that a staging misconfiguration cannot reach production data.

27.2 CI/CD pipeline

#	STAGE	DETAIL
1	Lint & type-check	ESLint/Prettier (TS); Ruff/mypy (Python)
2	Unit tests	Jest, pytest
3	Build	Vite (web), NestJS (api), Docker image (ai-service)
4	Integration tests	Testcontainers spin up Postgres/Redis
5	Security scan	Trivy, dependency audit
6	Push images	Tagged with commit SHA, pushed to ECR
7	Deploy to staging	ArgoCD auto-syncs the staging overlay on merge to `main`
8	Smoke tests	Playwright suite against staging
9	Manual approval	Required reviewer approval to promote
10	Deploy to production	ArgoCD syncs the production overlay with the approved tag

Deployment strategy is rolling for the core API and AI service, blue-green for the web build behind CloudFront. Every production deployment traces to a specific commit SHA and CI run — extending the audit trail from code to running workload.

27.3 Infrastructure

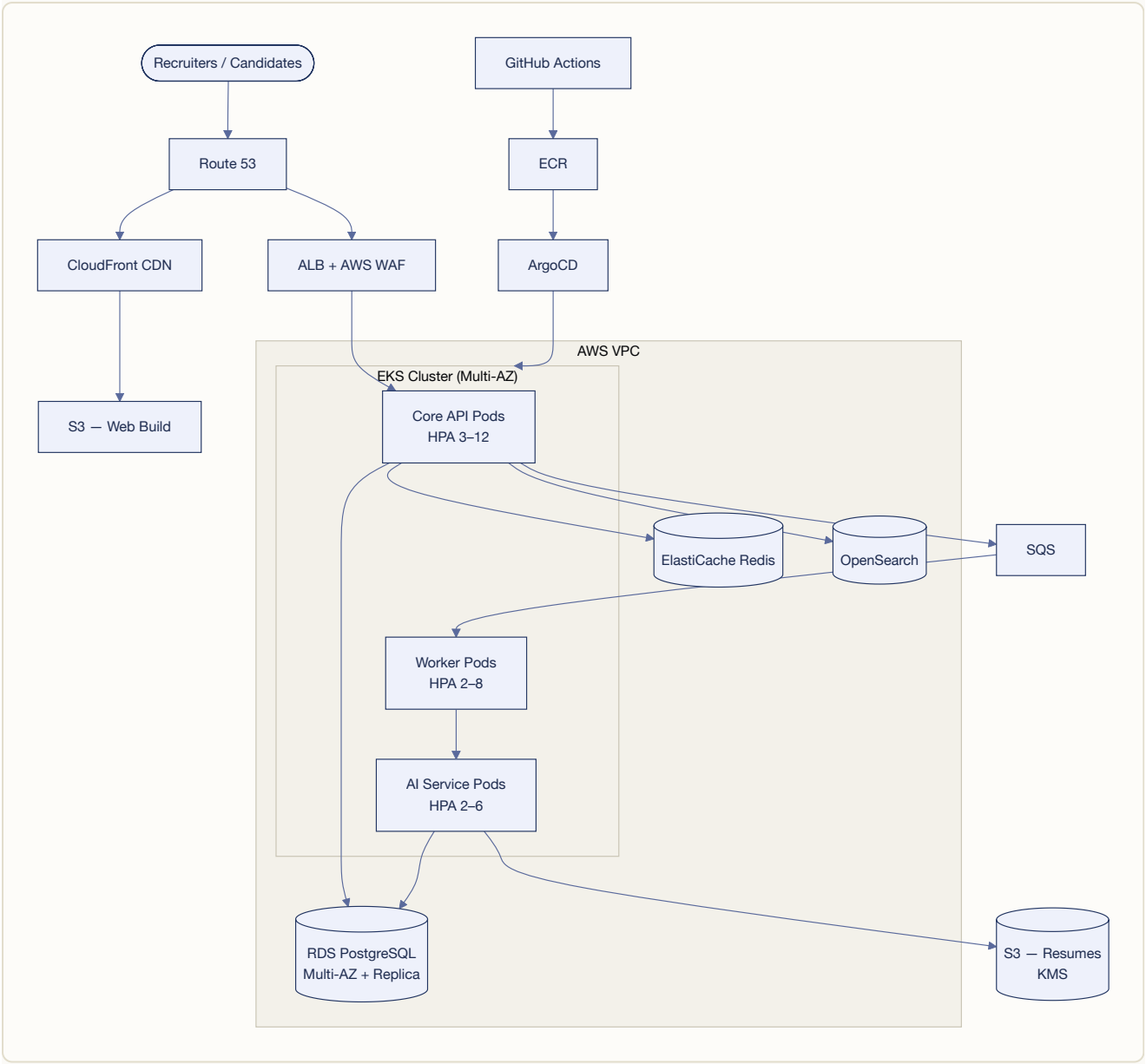


Figure 27.1 — Production deployment topology.

27.4 Scalability and performance

DIMENSION	APPROACH
Horizontal autoscaling	Kubernetes HPA on core API (CPU + request rate) and AI service (CPU + queue depth), with headroom to 3× the NFR3 target
Database	Read replica absorbs analytics queries; PgBouncer connection pooling prevents exhaustion under burst
Async absorption	The SQS-backed queue absorbs submission spikes without slowing the synchronous submit path; workers scale independently to drain backlog
Caching	Redis cache-aside for job listings and ranked lists, invalidated on write; CloudFront edge-caches the static build
Query performance	`EXPLAIN ANALYZE` review required on any query touching `applications` or `match_scores`; cursor pagination avoids `OFFSET` cost
AI inference	Embeddings batched where possible; models held warm in memory to avoid cold-start latency

27.5 Backup and recovery

CONTROL	SPECIFICATION
Database backup	RDS automated daily snapshots, point-in-time recovery, 35-day retention, cross-region copy
Object storage	S3 versioning on the resumes bucket; cross-region replication
Recovery objectives	RTO 4 hours; RPO 15 minutes (NFR9)
Restore drills	Quarterly — a snapshot is restored into an isolated environment and validated against a checklist
Migration safety	Destructive migrations deploy in two steps (stop writing → verify → drop in a later release) to preserve rollback

28. Risk Analysis

Table 28.1 — Risk register

ID	RISK	LIKELIHOOD	IMPACT	MITIGATION	OWNER
R-01	AI ranking produces disparate impact across protected groups	Medium	**High** — legal and reputational exposure	Mandatory pre-launch and quarterly bias testing; promotion gate with no override; human-in-the-loop on all screening decisions	AI Lead
R-02	Resume parsing accuracy varies across formats	High	Medium — degraded candidate and recruiter experience	Per-field confidence scoring with review flagging; OCR fallback; nightly golden-dataset evaluation	AI Lead
R-03	Recruiter distrust of AI recommendations suppresses adoption	Medium	Medium — BG-2 failure	Rationale shown for every score; full override preserved; override rate monitored as a trust signal	Product
R-04	Scope creep during development	High	Medium — schedule slippage	Scope locked in §14; new requests routed to §31 backlog	Product
R-05	Data privacy or security incident	Low	**High** — legal exposure, loss of trust	Encryption at rest and in transit; RBAC at API layer; secrets rotation; annual penetration test	Security
R-06	Third-party calendar/API instability	Medium	Low–Medium — scheduling disruption	Graceful degradation to manual scheduling (FR6.3)	Backend
R-07	Training data insufficient for a reliable re-ranker at launch	Medium	Medium — poor shortlist precision	Launch with semantic similarity and rule-weighted scoring; enable the learned re-	AI Lead

ID	RISK	LIKELIHOOD	IMPACT	MITIGATION	OWNER
				ranker only once labelled volume passes the evaluation gates	
R-08	Model drift degrades quality silently post-launch	Medium	Medium	PSI drift monitoring and override-rate regression alerts trigger retraining (§19.5)	AI Lead
R-09	Cross-tenant data leakage through a missed query scope	Low	**High**	`TenantGuard` applied globally at framework level rather than per-service; integration tests assert isolation	Backend

R-07, R-08, and R-09 were absent from both source documents and are added here: the first two are the standard failure modes of a learned ranking system at launch and in steady state, and the third is the characteristic severe defect of multi-tenant SaaS.

29. Timeline

Table 29.1 — Delivery phases

PHASE	DURATION	CUMULATIVE	KEY OUTPUTS
P1 — Requirements & Design	3 weeks	Week 3	Approved PRD; system architecture; UX designs; schema design
P2 — Core Platform Development	6 weeks	Week 9	Job management, application intake, pipeline, RBAC, audit logging
P3 — AI Screening Development	4 weeks	Week 13	Parsing pipeline, ranking, explainability, model registry integration
P4 — QA, Accessibility & Fairness Testing	3 weeks	Week 16	Full test suite green; WCAG audit; bias evaluation
P5 — Beta Release	2 weeks	Week 18	Limited release to design partners; baseline metrics captured
P6 — General Availability	1 week	Week 19	Production launch; monitoring and on-call live

Table 29.2 — Milestones

ID	MILESTONE	GATE
M1	Requirements sign-off	PRD approved by all stakeholders
M2	Design complete	Architecture and UX finalised
M3	Core platform build	Job management, intake, and pipeline functional end-to-end
M4	AI screening integration	Parsing and ranking integrated and tested
M5	Fairness & compliance validation	Bias testing and accessibility audit passed
M6	Beta release	Design-partner release live; baseline captured
M7	General availability	Full MVP launch

Critical path note. P3 depends on P2's application-intake completion, but the parsing pipeline can be developed in parallel against fixture data from week 5, compressing the effective critical path. The genuinely serial dependency is P4's fairness testing, which requires the completed P3 scoring pipeline and cannot be shortened without weakening M5.

30. Acceptance Criteria

The MVP is releasable only when **every** criterion below is satisfied. Criteria are grouped by type; none is discretionary.

30.1 Functional

- All functional requirements in §11 implemented and verified by test case.
- Recruiters can override any AI-generated score or rank, with the override reflected immediately in the pipeline view and recorded in the audit log.
- AI-generated rankings display a rationale in 100% of cases where a score is shown.
- Role-based access control verified across all four roles, including negative cases.
- Audit log export verified end-to-end: generate activity → export → reconcile against source records.

30.2 Quality & compliance

- Bias/fairness testing completed on the launch model, with all protected-group outcomes within the four-fifths threshold.
- WCAG 2.1 AA accessibility audit passed on all candidate- and recruiter-facing flows.
- Platform meets stated performance targets under load testing (10 s p95 parsing, 2 s p95 page load, 400 ms p95 scoring) at 3× launch scale.
- Security scan and penetration test findings resolved or formally risk-accepted.
- Legal review of the AI-in-hiring compliance posture completed.

30.3 Operational readiness

Table 30.1 — Production go-live checklist

#	ITEM
1	All environment secrets provisioned in Secrets Manager; no `.env` in production
2	Terraform plan reviewed and applied for the production account
3	Database migrations applied and verified against a production-sized dataset
4	Monitoring dashboards live and reviewed by on-call
5	Alert routing to PagerDuty/Slack verified with a synthetic alert
6	Backup restore drill completed successfully within the last quarter
7	Runbooks written for the top five anticipated incident types
8	On-call rotation staffed and escalation policy configured
9	Rollback procedure tested in staging for both application and database
10	Baseline time-to-shortlist captured from design partners (required to evaluate BG-1)

31. Future Scope

Table 31.1 — Post-v1 roadmap

ITEM	RATIONALE	DEPENDENCY
DEI analytics dashboard	Surfaces aggregate fairness reporting to customers, not only internal audit	Requires accumulated production volume for statistical significance
Chat-based recruiting assistant	Reduces navigation cost for infrequent hiring-manager users	Stable core API
Job board syndication	Expands top-of-funnel sourcing	Partner integrations
Offer management with e-signature	Closes the remaining manual gap in the hiring lifecycle	Third-party signature provider
Native mobile applications	Candidate-side convenience beyond responsive web	Sustained mobile traffic evidence
Multi-language resume parsing	Market expansion beyond English-language hiring	Labelled corpus per target language
Self-hosted deployment option	Serves data-residency-sensitive enterprise customers	Packaging and support model
Structured interview-kit generation	Improves scorecard comparability across interviewers	Scorecard usage data

32. Glossary

TERM	DEFINITION
Adverse impact	A substantially different rate of selection that works to the disadvantage of a protected group
ATS	Applicant Tracking System
Bi-encoder	A model architecture encoding two texts independently into vectors for similarity comparison, enabling precomputation and fast retrieval
Canary deployment	Releasing a change to a small traffic subset before full rollout
Composite score	The final calibrated candidate–job match score in [0,1]
Correlation ID	An identifier propagated across services and queues to trace a single logical request
Four-fifths rule	A regulatory heuristic: a selection rate for any group below 80% of the highest group's rate indicates possible adverse impact
Golden dataset	A frozen, human-labelled evaluation set used to detect regression between model versions
HPA	Horizontal Pod Autoscaler (Kubernetes)
IRSA	IAM Roles for Service Accounts — scoped AWS permissions per Kubernetes workload
NDCG	Normalised Discounted Cumulative Gain, a ranking-quality metric that rewards placing relevant items higher
NER	Named Entity Recognition
PII	Personally Identifiable Information
PSI	Population Stability Index, used to detect distribution drift
RBAC	Role-Based Access Control
RPO / RTO	Recovery Point Objective (tolerable data loss) / Recovery Time Objective (tolerable downtime)
Shadow mode	Running a model in production, scoring live traffic, without surfacing its output to users
SHAP	SHapley Additive exPlanations — per-prediction feature attribution

TERM	DEFINITION
Train/serve skew	Divergence between feature computation at training time and at inference time

33. References

1. U.S. Equal Employment Opportunity Commission. *Uniform Guidelines on Employee Selection Procedures*, 29 C.F.R. Part 1607.
2. New York City Department of Consumer and Worker Protection. *Local Law 144 of 2021: Automated Employment Decision Tools*.
3. European Parliament and Council. *Regulation (EU) 2016/679 — General Data Protection Regulation*, Articles 15 and 22.
4. Lundberg, S. M., & Lee, S.-I. (2017). *A Unified Approach to Interpreting Model Predictions*. Advances in Neural Information Processing Systems 30.
5. Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. EMNLP-IJCNLP.
6. Ke, G., et al. (2017). *LightGBM: A Highly Efficient Gradient Boosting Decision Tree*. NeurIPS 30.
7. Mitchell, M., et al. (2019). *Model Cards for Model Reporting*. ACM FAT*.
8. Sculley, D., et al. (2015). *Hidden Technical Debt in Machine Learning Systems*. NeurIPS 28.
9. OWASP Foundation. *OWASP Application Security Verification Standard (ASVS) v4.0*.
10. W3C. *Web Content Accessibility Guidelines (WCAG) 2.1*.
11. Wiggins, A. *The Twelve-Factor App*. 12factor.net.
12. Amazon Web Services. *AWS Well-Architected Framework*.

Appendix A — Supplementary Diagrams

A.1 Domain class model

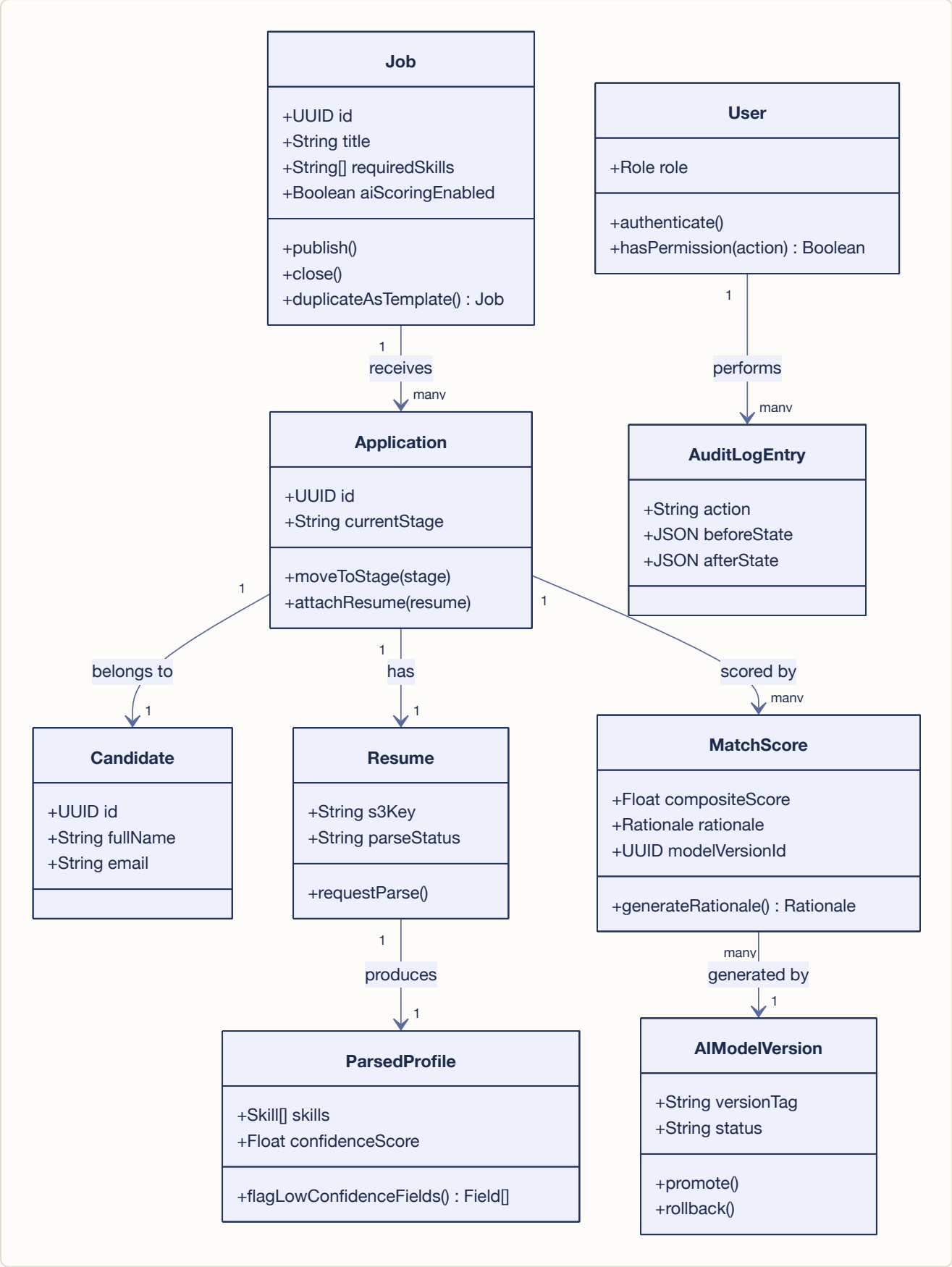


Figure A.1 — Domain class model for the Core API.

A.2 Application-to-shortlist sequence

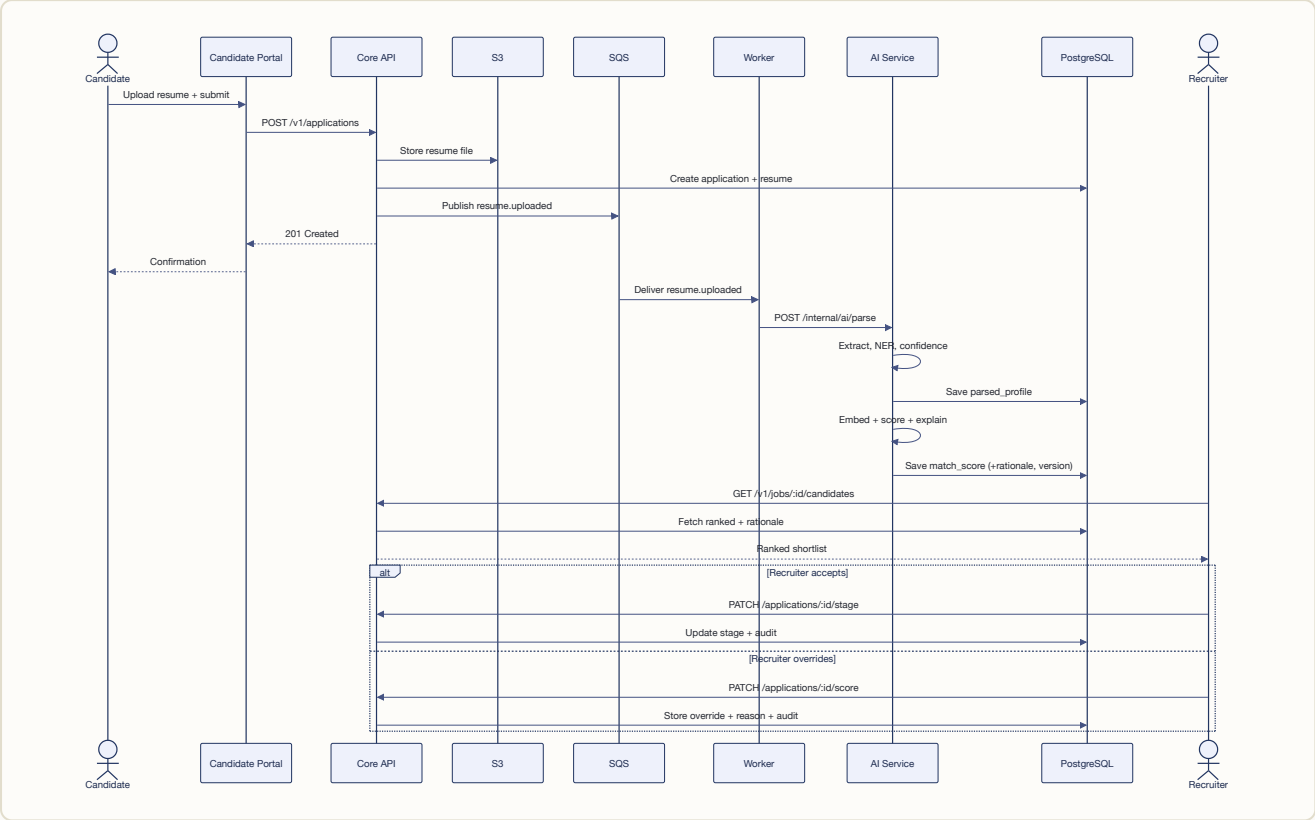


Figure A.2 — Sequence from application submission to recruiter decision.

End of Document

AI-Powered Recruitment & Applicant Tracking System — Final Master PRD v3.0 Riya Jaiswal · GD Goenka University · B.Tech Computer Science Engineering · 2026–27